

```
In [1]: # package importation
# data
import pandas as pd
import numpy as np

# plot
import matplotlib.pyplot as plt
from matplotlib import cm #Colormap
import matplotlib.patches as mpatches
import matplotlib.lines as mlines
import matplotlib.transforms as mtransforms
import matplotlib.gridspec as gridspec #Subplot in subplot
from matplotlib.ticker import MaxNLocator

import seaborn as sns

import scipy
from scipy.stats import ttest_ind as ttest

# data treatment
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from collections import Counter

In [2]: sns.set_style("darkgrid")

DATA_PATH = "../data/"
OUTPUT_PATH = '../output/'
```

Import

```
In [3]: df_syn = pd.read_csv(DATA_PATH + "synthetic_data.txt", sep=";")
df_syn.head()
```

Out[3]:

	ID	ID per year	Year	Dum_Default	Total equity total assets	Total reg cap ratio	Liquid assets total assets	Deposits mm funding growth	Expenses rev	Net int margin
0	13215466	132154662014	2014	0	14.910683	18.282517	3.077498	1616.302825	56.023078	3.710629
1	21776652	217766522002	2002	0	13.900000	19.100000	5.600000	60.000000	67.500000	6.200000
2	24093640	240936402012	2012	0	9.279741	14.820007	6.387157	-50.020364	53.888990	4.249898
3	18010425	180104252005	2005	0	10.020848	12.377526	4.335830	-37.427293	70.134845	3.835830
4	3606768	36067682009	2009	0	9.466163	21.222226	5.923254	146.421381	98.272408	3.207873

5 rows x 24 columns

```
In [4]: df_orig = pd.read_csv(DATA_PATH + "df_US.txt", sep=";")
df_orig.head()
```

Out[4]:

	FitchID	ID	Year	Bank	Country	Dum_Default	Total equity total assets	Total reg cap ratio	Liquid assets total assets	Deposits mm funding growth	...	Total noi earning! asset over T/
0	1012594	10125942000	2000	1867 Western Financial Corporation	UnitedStates	0	14.51	17.7	9.22	1338.0	...	0.06613
1	1199091	11990912000	2000	1880 Bank	UnitedStates	0	14.70	26.8	2.20	1640.0	...	0.05839
2	1201339	12013392000	2000	1st Bank & Trust	UnitedStates	0	8.80	14.2	6.20	80.0	...	0.07539
3	1202531	12025312000	2000	1st Bank of Sea Isle City	UnitedStates	0	7.10	12.2	5.20	1690.0	...	0.07063
4	1201905	12019052000	2000	1st Constitution Bank	UnitedStates	0	8.50	13.5	4.20	1230.0	...	0.05230

5 rows × 29 columns



```
In [6]: l_var = list(df_syn)[4:]
l_var == list(df_orig)[6:-3]
```

Out[6]: True

Stability test: difference in mean

- we test the significancy of the difference in mean and variance for each variable:
 - Student test for means
 - Fisher test for variances
- In each case: we look into the means and variances of a variable for the full sample, the sample in which Dum_Default scores 1 and the one in which Dum_Default scores 0. We campare original and synthetic data.

\$\rightarrow\$ Those tests tell us about the relevance of each variable ability to split the sample when looking into the default variable.

Full sample

```
In [7]: def custom_sort(key):
        return d_full_mean[key][1]

# full sample
d_full_mean = {}
for var in l_var:
    a = df_syn[var]
    b = df_orig[var]
    iz = ttest(a,b)

    d_full_mean[var] = [np.round(iz[0],3),np.round(iz[1],2)]

# Trier les clés du dictionnaire en fonction du premier élément de chaque clé
sorted_keys = sorted(d_full_mean, key=custom_sort)

# Afficher les éléments du dictionnaire dans l'ordre croissant du premier élément de chaque clé
print(f"Student's statistic for difference in mean: \n")
for var in sorted_keys:
    print(f"{var}: {d_full_mean[var][0]} and p-value {d_full_mean[var][1]}")
```

Student's statistic for difference in mean:

Liquid assets total assets: -4.618 and p-value 0.0
Operating roaa: -3.108 and p-value 0.0
Tax expense over TA: -2.816 and p-value 0.0
Total desposits over TA: 3.356 and p-value 0.0
Total funding over TA: 3.287 and p-value 0.0
log_Avg earning assets over TA: 7.573 and p-value 0.0
log_Bus volume over TA: 3.0 and p-value 0.0
log_Liquid assets deposits mm funding: -4.916 and p-value 0.0
log_Total non int exp over TA: -2.864 and p-value 0.0
Total equity total assets: -2.24 and p-value 0.03
Total oper expense over TA: -1.989 and p-value 0.05
Total reg cap ratio: -1.77 and p-value 0.08
log_Total assets: 1.496 and p-value 0.13
Total int exp over TA: -1.441 and p-value 0.15
Total non earning assets over TA: -0.988 and p-value 0.32
Non earning assets gs over TA: -0.984 and p-value 0.33
Net int margin: -0.897 and p-value 0.37
Deposits mm funding growth: 0.208 and p-value 0.84
Net loans total assets: 0.206 and p-value 0.84
Expenses rev: -0.186 and p-value 0.85

```
In [8]: sns.histplot(df_orig['Expenses rev'], kde=True, bins=int(180/5), color = 'darkblue',ec="black",linewidth=0.5)
plt.show()

sns.histplot(df_syn['Expenses rev'], kde=True, bins=int(180/5), color = 'red',ec="black",linewidth=0.5)
plt.show()

# Compared histogram: expense revenue

fig, ax1 = plt.subplots()

ax1.set_xlabel('Expenses Revenue',fontsize=20)
ax1.hist(df_orig['Expenses rev'],
        , color='blue', edgecolor='black', bins=int(100), alpha = 0.6)
ax1.set_ylabel('Number of original points',fontsize=20, color='b')

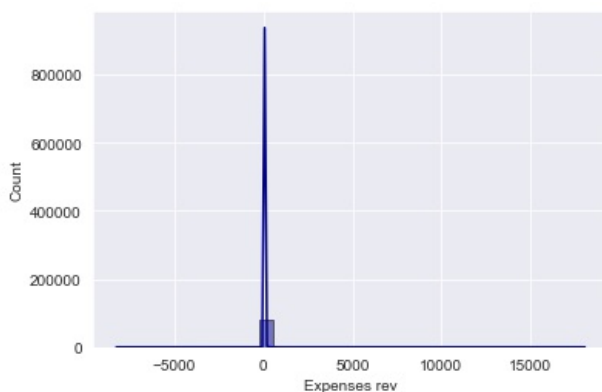
ax2 = ax1.twinx() # instantiate a second axes that shares the same x-axis

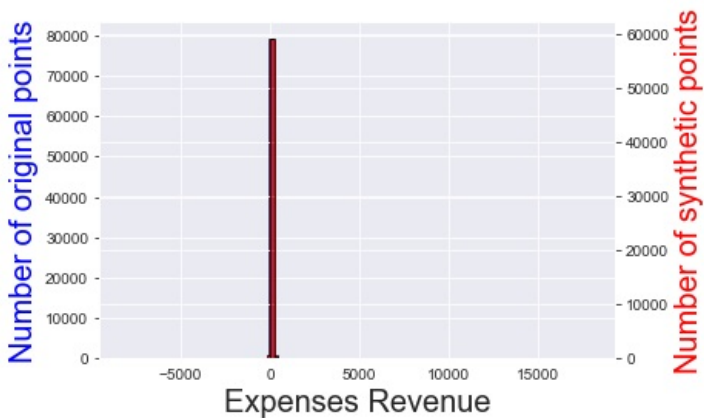
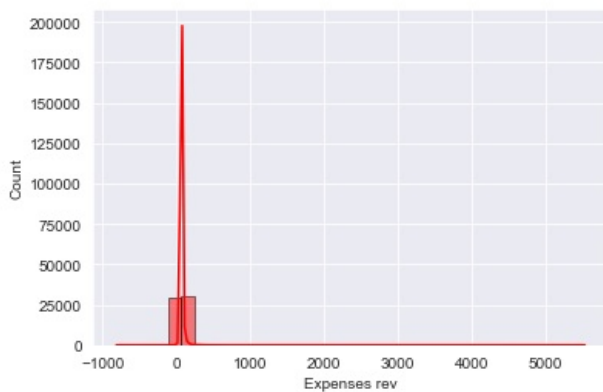
ax2.set_ylabel('Number of synthetic points',fontsize=20, color='r') # we already handled the x-label with ax1
ax2.hist(df_syn['Expenses rev'],
        , color='r', edgecolor='black', bins=int(30), alpha = 0.8)

#plt.title('TE/TA - Distribution, all banks',fontsize = 22)

#plt.savefig(path_output + 'distribution_TE_over_TA_us.png')

plt.show()
```





Zero sample (non default banks)

```
In [9]: def custom_sort(key):
        return d_zero_mean[key][1]

# zero sample
d_zero_mean = {}
for var in l_var:
    a = df_syn[df_syn.Dum_Default == 0][var]
    b = df_orig[df_orig.Dum_Default == 0][var]
    iz = ttest(a,b)

    d_zero_mean[var] = [np.round(iz[0],3),np.round(iz[1],2)]

# Trier les clés du dictionnaire en fonction du premier élément de chaque clé
sorted_keys = sorted(d_zero_mean, key=custom_sort)

# Afficher les éléments du dictionnaire dans l'ordre croissant du premier élément de chaque clé
print(f"Student's statistic for difference in mean: \n")
for var in sorted_keys:
    print(f"{var}: {d_zero_mean[var][0]} and p-value {d_zero_mean[var][1]}")
```

Student's statistic for difference in mean:

Liquid assets total assets: -4.914 and p-value 0.0
Total desoposits over TA: 3.11 and p-value 0.0
log_Avg earning assets over TA: 7.401 and p-value 0.0
log_Bus volume over TA: 3.139 and p-value 0.0
log_Liquid assets deposits mm funding: -5.144 and p-value 0.0
log_Total non int exp over TA: -3.09 and p-value 0.0
Operating roaa: -2.509 and p-value 0.01
Tax expense over TA: -2.614 and p-value 0.01
Total funding over TA: 2.727 and p-value 0.01
Total oper expense over TA: -2.43 and p-value 0.02
Total int exp over TA: -1.839 and p-value 0.07
log_Total assets: 1.704 and p-value 0.09
Total equity total assets: -1.578 and p-value 0.11
Non earning assets gs over TA: -1.549 and p-value 0.12
Total non earning assets over TA: -1.553 and p-value 0.12
Total reg cap ratio: -1.445 and p-value 0.15
Expenses rev: -1.08 and p-value 0.28
Net int margin: -0.586 and p-value 0.56
Deposits mm funding growth: 0.212 and p-value 0.83
Net loans total assets: 0.099 and p-value 0.92

One sample (default banks)

```
In [10]: def custom_sort(key):
        return d_one_mean[key][1]

# one sample
d_one_mean = {}
for var in l_var:
    a = df_syn[df_syn.Dum_Default == 1][var]
    b = df_orig[df_orig.Dum_Default == 1][var]
    iz = ttest(a,b)

    d_one_mean[var] = [np.round(iz[0],3),np.round(iz[1],2)]

# Trier les clés du dictionnaire en fonction du premier élément de chaque clé
sorted_keys = sorted(d_one_mean, key=custom_sort)

# Afficher les éléments du dictionnaire dans l'ordre croissant du premier élément de chaque clé
print(f"Student's statistic for difference in mean: \n")
for var in sorted_keys:
    print(f"{var}: {d_one_mean[var][0]} and p-value {d_one_mean[var][1]}")
```

Student's statistic for difference in mean:

log_Avg earning assets over TA: 0.784 and p-value 0.43
log_Total assets: -0.745 and p-value 0.46
Total desoposits over TA: 0.724 and p-value 0.47
log_Bus volume over TA: -0.582 and p-value 0.56
Total oper expense over TA: 0.468 and p-value 0.64
Operating roaa: -0.427 and p-value 0.67
Liquid assets total assets: 0.368 and p-value 0.71
Net loans total assets: -0.348 and p-value 0.73
Deposits mm funding growth: -0.314 and p-value 0.75
Total funding over TA: -0.277 and p-value 0.78
Total int exp over TA: 0.202 and p-value 0.84
log_Total non int exp over TA: 0.167 and p-value 0.87
Tax expense over TA: 0.147 and p-value 0.88
Total equity total assets: 0.141 and p-value 0.89
Total reg cap ratio: 0.142 and p-value 0.89
Expenses rev: -0.116 and p-value 0.91
log_Liquid assets deposits mm funding: -0.091 and p-value 0.93
Net int margin: 0.075 and p-value 0.94
Non earning assets gs over TA: -0.009 and p-value 0.99
Total non earning assets over TA: -0.009 and p-value 0.99

Stability test: difference in variance

```
In [11]: #define F-test function
def f_test(x, y):
    x = np.array(x)
    y = np.array(y)
    f = np.var(x, ddof=1)/np.var(y, ddof=1) #calculate F test statistic
    dfn = x.size-1 #define degrees of freedom numerator
```

```

dfd = y.size-1 #define degrees of freedom denominator
p = 1-scipy.stats.f.cdf(f, dfn, dfd) #find p-value of F test statistic
return f, p

```

Full sample

```

In [12]: def custom_sort(key):
          return d_full_var[key][1]

# full sample
d_full_var = {}
for var in l_var:
    a = df_syn[var]
    b = df_orig[var]
    iz = f_test(a,b)

    d_full_var[var] = [np.round(iz[0],3),np.round(iz[1],2)]

# Trier les clés du dictionnaire en fonction du premier élément de chaque clé
sorted_keys = sorted(d_full_var, key=custom_sort)

# Afficher les éléments du dictionnaire dans l'ordre croissant du premier élément de chaque clé
print(f"Student's statistic for difference in variance: \n")
for var in sorted_keys:
    print(f"{var}: {d_full_var[var][0]} and p-value {d_full_var[var][1]}")

```

Student's statistic for difference in variance:

```

Total equity total assets: 0.743 and p-value 1.0
Total reg cap ratio: 0.583 and p-value 1.0
Liquid assets total assets: 0.73 and p-value 1.0
Deposits mm funding growth: 0.797 and p-value 1.0
Expenses rev: 0.322 and p-value 1.0
Net int margin: 0.771 and p-value 1.0
Net loans total assets: 0.84 and p-value 1.0
Non earning assets gs over TA: 0.723 and p-value 1.0
Operating roaa: 0.146 and p-value 1.0
Tax expense over TA: 0.209 and p-value 1.0
Total desposits over TA: 0.766 and p-value 1.0
Total funding over TA: 0.681 and p-value 1.0
Total int exp over TA: 0.699 and p-value 1.0
Total non earning assets over TA: 0.723 and p-value 1.0
Total oper expense over TA: 0.655 and p-value 1.0
log_Avg earning assets over TA: 0.612 and p-value 1.0
log_Bus volume over TA: 0.975 and p-value 1.0
log_Liquid assets deposits mm funding: 0.75 and p-value 1.0
log_Total assets: 0.931 and p-value 1.0
log_Total non int exp over TA: 0.643 and p-value 1.0

```

Zero sample (non default banks)

```

In [13]: def custom_sort(key):
          return d_zero_var[key][1]

# zero sample
d_zero_var = {}
for var in l_var:
    a = df_syn[df_syn.Dum_Default == 0][var]
    b = df_orig[df_orig.Dum_Default == 0][var]
    iz = f_test(a,b)

    d_zero_var[var] = [np.round(iz[0],3),np.round(iz[1],2)]

# Trier les clés du dictionnaire en fonction du premier élément de chaque clé
sorted_keys = sorted(d_zero_var, key=custom_sort)

# Afficher les éléments du dictionnaire dans l'ordre croissant du premier élément de chaque clé
print(f"Student's statistic for difference in var: \n")
for var in sorted_keys:
    print(f"{var}: {d_zero_var[var][0]} and p-value {d_zero_var[var][1]}")

```

Student's statistic for difference in var:

Total equity total assets: 0.738 and p-value 1.0
Total reg cap ratio: 0.583 and p-value 1.0
Liquid assets total assets: 0.729 and p-value 1.0
Deposits mm funding growth: 0.799 and p-value 1.0
Expenses rev: 0.191 and p-value 1.0
Net int margin: 0.771 and p-value 1.0
Net loans total assets: 0.842 and p-value 1.0
Non earning assets gs over TA: 0.717 and p-value 1.0
Operating roaa: 0.137 and p-value 1.0
Tax expense over TA: 0.207 and p-value 1.0
Total desoposits over TA: 0.766 and p-value 1.0
Total funding over TA: 0.677 and p-value 1.0
Total int exp over TA: 0.697 and p-value 1.0
Total non earning assets over TA: 0.717 and p-value 1.0
Total oper expense over TA: 0.652 and p-value 1.0
log_Avg earning assets over TA: 0.61 and p-value 1.0
log_Bus volume over TA: 0.977 and p-value 1.0
log_Liquid assets deposits mm funding: 0.749 and p-value 1.0
log_Total assets: 0.934 and p-value 1.0
log_Total non int exp over TA: 0.642 and p-value 1.0

One sample (default banks)

```
In [14]: def custom_sort(key):
          return d_one_var[key][1]

# one sample
d_one_var = {}
for var in l_var:
    a = df_syn[df_syn.Dum_Default == 1][var]
    b = df_orig[df_orig.Dum_Default == 1][var]
    iz = f_test(a,b)

    d_one_var[var] = [np.round(iz[0],3),np.round(iz[1],2)]

# Trier les clés du dictionnaire en fonction du premier élément de chaque clé
sorted_keys = sorted(d_one_var, key=custom_sort)

# Afficher les éléments du dictionnaire dans l'ordre croissant du premier élément de chaque clé
print(f"Student's statistic for difference in var: \n")
for var in sorted_keys:
    print(f"{var}: {d_one_var[var][0]} and p-value {d_one_var[var][1]}")
```

Student's statistic for difference in var:

Total oper expense over TA: 0.791 and p-value 0.99
Total equity total assets: 0.659 and p-value 1.0
Total reg cap ratio: 0.648 and p-value 1.0
Liquid assets total assets: 0.721 and p-value 1.0
Deposits mm funding growth: 0.687 and p-value 1.0
Expenses rev: 0.629 and p-value 1.0
Net int margin: 0.642 and p-value 1.0
Net loans total assets: 0.632 and p-value 1.0
Non earning assets gs over TA: 0.682 and p-value 1.0
Operating roaa: 0.771 and p-value 1.0
Tax expense over TA: 0.687 and p-value 1.0
Total desoposits over TA: 0.678 and p-value 1.0
Total funding over TA: 0.634 and p-value 1.0
Total int exp over TA: 0.667 and p-value 1.0
Total non earning assets over TA: 0.682 and p-value 1.0
log_Avg earning assets over TA: 0.718 and p-value 1.0
log_Bus volume over TA: 0.537 and p-value 1.0
log_Liquid assets deposits mm funding: 0.729 and p-value 1.0
log_Total assets: 0.608 and p-value 1.0
log_Total non int exp over TA: 0.778 and p-value 1.0

Stability tests: removing outliers

```
In [15]: # function for winsorisation: choice for removing upward values
def remove_outliers(df,var,pct,up=True):
    q = df[var].quantile(1-pct)
    df_filtred = df[df[var]<=q]
    n_removed = df.shape[0] - df_filtred.shape[0]

    if not up :
        q0 = df[var].quantile(pct)
```

```

q1 = df[var].quantile(1-pct)
df_filtred = df[(df[var]>=q0)&(df[var]<=q1)]
n_removed = df.shape[0] - df_filtred.shape[0]

return df_filtred, n_removed

```

```

In [16]: # retrieving variables
def custom_sort(key):
    return d_full_mean[key][1]

# full sample
d_full_mean = {}
for var in l_var:
    a = df_syn[var]
    b = df_orig[var]
    iz = ttest(a,b)

    d_full_mean[var] = [np.round(iz[0],3),np.round(iz[1],2)]

l_mean_out = []
sorted_keys = sorted(d_full_var, key=custom_sort)
for var in sorted_keys :
    if d_full_mean[var][1]>0.05 :
        l_mean_out.append(var)

```

```

In [17]: pct = 0.05

def custom_sort(key):
    return d_full_mean[key][1]

# full sample
d_full_mean = {}
for var in l_mean_out:
    a = remove_outliers(df_syn,var,pct,False)[0][var]
    b = remove_outliers(df_orig,var,pct,False)[0][var]
    iz = ttest(a,b)

    d_full_mean[var] = [np.round(iz[0],3),np.round(iz[1],2)]

# Trier les clés du dictionnaire en fonction du premier élément de chaque clé
sorted_keys = sorted(d_full_mean, key=custom_sort)

# Afficher les éléments du dictionnaire dans l'ordre croissant du premier élément de chaque clé
print(f"Student's statistic for difference in mean: \n")
for var in sorted_keys:
    print(f"{var}: {d_full_mean[var][0]} and p-value {d_full_mean[var][1]}")

```

Student's statistic for difference in mean:

Total reg cap ratio: 4.457 and p-value 0.0
 Total non earning assets over TA: 5.992 and p-value 0.0
 Non earning assets gs over TA: 5.999 and p-value 0.0
 Expenses rev: 1.946 and p-value 0.05
 Total int exp over TA: 1.707 and p-value 0.09
 Deposits mm funding growth: -1.522 and p-value 0.13
 Net loans total assets: -1.392 and p-value 0.16
 log_Total assets: 0.731 and p-value 0.46
 Net int margin: -0.119 and p-value 0.91

```

In [18]: pct = 0.05

def custom_sort(key):
    return d_full_var[key][1]

# full sample
d_full_var = {}
for var in l_var:
    a = remove_outliers(df_syn,var,pct,False)[0][var]
    b = remove_outliers(df_orig,var,pct,False)[0][var]
    iz = f_test(a,b)

    d_full_var[var] = [np.round(iz[0],3),np.round(iz[1],2)]

# Trier les clés du dictionnaire en fonction du premier élément de chaque clé
sorted_keys = sorted(d_full_var, key=custom_sort)

```



```
# Afficher les éléments du dictionnaire dans l'ordre croissant du premier élément de chaque clé
print(f"Student's statistic for difference in variance: \n")
for var in sorted_keys:
    print(f"{var}: {d_full_var[var][0]} and p-value {d_full_var[var][1]}")
```

Student's statistic for difference in variance:

Total equity total assets: 0.829 and p-value 1.0
 Total reg cap ratio: 0.886 and p-value 1.0
 Liquid assets total assets: 0.784 and p-value 1.0
 Deposits mm funding growth: 0.727 and p-value 1.0
 Expenses rev: 0.869 and p-value 1.0
 Net int margin: 0.833 and p-value 1.0
 Net loans total assets: 0.828 and p-value 1.0
 Non earning assets gs over TA: 0.772 and p-value 1.0
 Operating roaa: 0.85 and p-value 1.0
 Tax expense over TA: 0.83 and p-value 1.0
 Total desposits over TA: 0.784 and p-value 1.0
 Total funding over TA: 0.839 and p-value 1.0
 Total int exp over TA: 0.664 and p-value 1.0
 Total non earning assets over TA: 0.772 and p-value 1.0
 Total oper expense over TA: 0.88 and p-value 1.0
 log_Avg earning assets over TA: 0.721 and p-value 1.0
 log_Bus volume over TA: 0.956 and p-value 1.0
 log_Liquid assets deposits mm funding: 0.773 and p-value 1.0
 log_Total assets: 0.913 and p-value 1.0
 log_Total non int exp over TA: 0.866 and p-value 1.0

```
In [23]: for var in l_var:
          print(f"Variance for {var} is (synth vs original): {np.round(np.var(df_syn[var]),2)}, {np.round(np.var(df_o
```

Variance for Total equity total assets is (synth vs original): 16.92, 22.76
 Variance for Total reg cap ratio is (synth vs original): 172.06, 295.05
 Variance for Liquid assets total assets is (synth vs original): 44.11, 60.41
 Variance for Deposits mm funding growth is (synth vs original): 531351266899.57, 666649633600.11
 Variance for Expenses rev is (synth vs original): 2800.86, 8691.9
 Variance for Net int margin is (synth vs original): 1.23, 1.59
 Variance for Net loans total assets is (synth vs original): 197.12, 234.74
 Variance for Non earning assets gs over TA is (synth vs original): 0.0, 0.0
 Variance for Operating roaa is (synth vs original): 4.46, 30.5
 Variance for Tax expense over TA is (synth vs original): 0.0, 0.0
 Variance for Total desposits over TA is (synth vs original): 0.01, 0.01
 Variance for Total funding over TA is (synth vs original): 0.0, 0.0
 Variance for Total int exp over TA is (synth vs original): 0.0, 0.0
 Variance for Total non earning assets over TA is (synth vs original): 0.0, 0.0
 Variance for Total oper expense over TA is (synth vs original): 0.0, 0.0
 Variance for log_Avg earning assets over TA is (synth vs original): 0.0, 0.0
 Variance for log_Bus volume over TA is (synth vs original): 0.03, 0.03
 Variance for log_Liquid assets deposits mm funding is (synth vs original): 0.38, 0.51
 Variance for log_Total assets is (synth vs original): 1.93, 2.07
 Variance for log_Total non int exp over TA is (synth vs original): 0.0, 0.0

In []: